Cost-Based SQL Query Optimizer

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a SQL query optimizer that answers the central question every real database engine has to answer: given a SQL query that joins multiple tables and filters them with predicates, in what order should the joins be performed and which rewrites should be applied first, to make the query run as fast as possible? The same query has many possible execution plans — some run in milliseconds, others take minutes — and the optimizer's job is to pick a good one without trying them all.

This project differs from the simpler version that other classes assign in three specific ways. First, the supported query subset includes joins of up to four tables, not just two; this is the size at which join ordering starts to matter dramatically and where the optimizer's choices are clearly visible in the benchmark numbers. Second, the optimizer is driven by an actual cost model: it maintains catalog statistics for every table and column (row counts, distinct values, value ranges) and uses standard cardinality estimation formulas to predict how big each intermediate result will be. Third, the optimizer applies a set of rewrite rules in addition to choosing a join order; you will implement four rules that any production optimizer also implements, plus the Selinger-style dynamic programming algorithm for join ordering, the same algorithm that has been at the core of System R, PostgreSQL, and DB2 since 1979.

The shape of the project is: a parser that accepts a small SQL subset, a catalog that holds table statistics, a logical-plan representation, a rule-based rewriter that applies four cleanup transformations, a join-order search that uses dynamic programming over a cost function, a materialized executor that runs the chosen plan, and a benchmark that compares the optimizer's chosen plans against several baselines (no optimizer at all, rewrites only without join ordering, join ordering only without rewrites). The benchmark is the headline deliverable: it makes the optimizer's value visible by showing speedups in the 100× to 1000× range on adversarial queries and quantifies the contribution of each component.

You are not building a complete SQL database. There are no transactions, no indexes, no concurrent queries, and no SQL features beyond the supported subset (which is precisely defined in Section 5). The executor is the same materialized model used in the simpler version of this project: each operator is a function that takes a list of rows and returns a list of rows, with no Volcano iterators, no pipelining, and no parallelism. The reason for keeping the executor simple is that the project is about the optimizer; a sophisticated executor would compete for time without teaching anything new about query planning.

This project is one of the harder ones in Project 02. The difficulty is concentrated in two places: cardinality estimation (cost models that produce wildly wrong estimates make the optimizer choose worse plans than no optimization at all, and the chaos test will show this), and the join-ordering DP (the algorithm is short but indexing into the bitmap of subsets needs to be exact). Plan time for instrumenting your cost estimates against actual runtime sizes during development; this is what real database engineers do, and your design document will require you to discuss your estimates' accuracy.

2 What the Final Product Looks Like

When your project is complete, you can compile the binary, load the benchmark dataset, and watch the optimizer make visibly different plan choices than the unoptimized baseline.

```
$ make
$ ./qopt --data ./benchdata
qopt: opened catalog with 4 tables
  customers      10,000 rows  (id INT PK, name TEXT, country TEXT, age INT)
  orders        500,000 rows  (id INT PK, customer_id INT, total DOUBLE,
                               year INT, status TEXT)
  line_items  2,000,000 rows  (order_id INT, product_id INT, qty INT,
                               price DOUBLE)
  products       50,000 rows  (id INT PK, name TEXT, category TEXT,
                               supplier_id INT)
```

```
qopt: stats loaded for 4 tables, 16 columns

qopt> SELECT customers.name, SUM(line_items.qty * line_items.price)
      FROM customers, orders, line_items
      WHERE customers.id = orders.customer_id
        AND orders.id = line_items.order_id
        AND customers.country = 'PK'
        AND orders.year = 2024;

--- WITHOUT optimizer (left-deep, FROM order, no rewrites) ---
plan:
  HashJoin(orders.id = line_items.order_id)
    HashJoin(customers.id = orders.customer_id)
      Filter(country='PK' AND year=2024)
        CrossProduct
          SeqScan(customers)              [10K rows]
          SeqScan(orders)                 [500K rows -> 5B intermediate]
      SeqScan(line_items)
estimated cost: 5,247,180,200
actual time: 12.83 seconds, 412 result rows

--- WITH optimizer (rules + cost-based DP join ordering) ---
plan:
  HashJoin(orders.id = line_items.order_id)
    HashJoin(customers.id = orders.customer_id)
      Filter(country='PK')
        SeqScan(customers)               [est 41 rows, actual 39]
      Filter(year=2024)
        SeqScan(orders)                  [est 71K rows, actual 71K]
    SeqScan(line_items)                  [reads on demand via hash probe]
estimated cost: 2,840,120
actual time: 0.124 seconds, 412 result rows

speedup: 103.5x
plan cost ratio: 1847x
estimate error: cust 5%, orders 0.4%, joined 7%

qopt> \stats
queries executed:    1
optimizer wins:      1 (avg speedup 103.5x)
average plan time:   2.1ms
average exec time:   124ms
```

The defining moment of this session is the side-by-side plan comparison. The unoptimized version computes a 10K × 500K cross product before filtering — 5 billion intermediate rows — because it follows the FROM clause order naively. The optimized version filters `customers` down to 41 rows first (using the predicate `country = 'PK'`), then joins to a filtered `orders` (71K rows), then probes `line_items` via a hash join. The plan touches three orders of magnitude less data and runs three orders of magnitude faster. The optimizer's cost estimate (about 1800× better than the baseline) accurately predicted that the new plan would be much better, and the runtime measurement confirms it.

3 Architectural Overview

The system has six components. Keep them strictly separated.

```
                    [ SQL query string ]
                            |
                            v
                    +---------------+
                    |  Parser       |  produces a logical plan tree
                    +-------+-------+
                            |
                            v
                    +---------------+
```

```
                    |  Catalog       |  table schemas, row counts,
                    |  (Section 6)   |  per-column statistics
                    +-------+-------+
                            |
                            v
                    +---------------+
                    |  Rule          |  predicate pushdown,
                    |  rewriter      |  projection pushdown,
                    |  (Section 7)   |  constant folding, join swap
                    +-------+-------+
                            |
                            v
                    +---------------+
                    |  Cost model    |  cardinality + cost estimates
                    |  (Section 8)   |  using the catalog stats
                    +-------+-------+
                            |
                            v
                    +---------------+
                    |  Join-order    |  Selinger DP over subsets
                    |  search        |  of base tables
                    |  (Section 9)   |
                    +-------+-------+
                            |
                            v
                    +---------------+
                    |  Executor      |  materialized operator model
                    |  (Section 10)  |  (same as simpler version)
                    +---------------+
```

The pipeline is strictly top to bottom: the parser produces a logical plan, the rewriter cleans it up, the cost model and join-order search work together to choose a physical plan, and the executor runs it. There are no back-edges. This is the textbook structure of every real query optimizer, just stripped down.

4 The Supported SQL Subset

The grammar is precisely:

```
query     := SELECT select_list
             FROM table (, table)*
             [WHERE pred (AND pred)*]
             [GROUP BY column]
             [LIMIT integer]

select_list := * | expr (, expr)*
expr        := column
             | aggregate(column)
             | column op column        (e.g., qty * price)
             | literal
aggregate   := SUM | COUNT | AVG | MIN | MAX

pred        := column op literal       (e.g., year = 2024)
             | column op column        (e.g., customers.id = orders.customer_id)
op          := = | < | <= | > | >= | !=
```

Multiple tables in the FROM clause are joined: every pair of tables in the FROM produces a join, and the join condition is whichever WHERE predicate compares columns from those two tables. Predicates that compare a column to a literal (like `year = 2024`) are filters. Predicates with `AND` are independent; `OR` is not supported. Subqueries, ORDER BY, HAVING, DISTINCT, and OUTER JOIN are all out of scope.

The parser is a hand-written recursive-descent parser, the same shape as in the simpler version of this project. It is about 250 lines of C and produces a small logical-plan tree of the form:

```
LogicalPlan := Scan(table)
             | Filter(pred, child)
             | Join(condition, left, right)          (binary, hash join)
             | Project(exprs, child)
             | GroupBy(column, agg, child)
             | Limit(n, child)
```

A query like `SELECT * FROM a, b WHERE a.x = b.y AND a.z > 10` parses initially into something like `Project(*, Filter(a.x=b.y AND a.z>10, Join(true, Scan(a), Scan(b))))` — a deliberately bad starting plan with a no-condition join (effectively a cross product) and all predicates piled on top. The optimizer's job is to turn this into a good plan.

5 The Catalog and Statistics

Every real optimizer relies on a catalog of metadata about each table. For this project, the catalog holds:

5.1 Per Table

- `name`: the table's name.
- `columns`: an ordered list of (name, type) pairs.
- `row_count`: the number of rows in the table.

5.2 Per Column

- `distinct_count`: the number of distinct values in the column.
- `min_value` and `max_value`: the smallest and largest values seen (only meaningful for numeric columns; for strings, leave them empty and use a fallback in the estimator).
- `null_count`: the number of NULL values. For this project, the data is null-free, so this is always 0; the field exists for completeness.

These are the standard "base statistics" that every textbook cost model uses. They are deliberately small; per-column histograms (which are vastly more accurate but harder to implement) are an explicit bonus.

5.3 Building the Catalog

At startup, the engine reads each table's CSV file and computes the statistics in a single pass. This takes a few seconds for the benchmark dataset and produces a small `catalog.json` file that is cached for subsequent runs.

5.4 Why Statistics Matter

The cost model in Section 8 uses these statistics to estimate intermediate result sizes. A wrong estimate produces a wrong cost, which makes the optimizer pick a wrong plan. The classic failure case: if the optimizer thinks a filter is selective (will keep only 1% of rows) when it is actually unselective (keeps 50%), it will join the filtered table early in the plan, but the resulting intermediate is much bigger than predicted and the rest of the plan is suboptimal. Your benchmark must include cases where your estimates are accurate (the optimizer wins) and the design document must discuss any cases where they are not.

6 The Rule Rewriter

Before doing any join-order search, the optimizer applies a fixed set of rewrite rules. Each rule is a function `rule(plan) -> plan` that returns a new plan with the rule applied as many times as it can be (typically using a recursive walk of the tree). The optimizer applies rules in a fixed order until none of them changes the plan further (a fixed point).

You will implement exactly four rules:

6.1 Predicate Pushdown

A `Filter` sitting on top of a `Join` can be split, with each conjunct pushed down to whichever child can evaluate it. The conjunct `a.country = 'PK'` on top of `Join(Scan(a), Scan(b))` becomes `Join(Filter(a.country Scan(a)), Scan(b))`. Conjuncts that reference both sides of the join (the join condition itself, like `a.id`

`= b.customer_id`) stay attached to the join.

This is the most important rewrite. It is the one that produces the dramatic speedups you saw in the example session. Implementing it correctly requires distinguishing single-table conjuncts (push down) from multi-table conjuncts (keep as the join condition or as a higher filter).

6.2 Projection Pushdown

A `Project` that selects only some columns can be pushed down through joins and filters, so that intermediate results carry only the columns that downstream operators actually need. `Project([a.name], Join(Scan(a), Scan(b)))` where `b`'s columns are not used downstream becomes `Project([a.name], Join(Project([a.id, a.name], Scan(a)), Project([b.id], Scan(b))))` — each scan now only reads the columns that participate in the join condition or the final select list.

Computing which columns are "needed" requires a small upward analysis: walk the tree, collecting the set of column references in the SELECT list, in any predicate, and in every join condition. Push this set as a constraint on each child. Test this rule carefully; an off-by-one will produce a plan that runs but returns wrong data.

6.3 Constant Folding

A predicate like `2024 = 2024` or `1 < 2` can be evaluated at planning time and replaced with `TRUE` or `FALSE`. A `TRUE` conjunct can be removed; a `FALSE` conjunct turns the entire filter into "no rows" (which the optimizer can use to skip the rest of the plan entirely). Compound expressions like `2024 = year` stay; only fully literal expressions fold.

This rule is small but valuable because it interacts with predicate pushdown: a predicate that reduces to `FALSE` after folding can prune entire branches. Test this case.

6.4 Join Input Swap

The hash join builds a hash table on its left input and probes with its right input. For a hash join to be efficient, the build side should be the smaller relation. After cost estimation has run, the optimizer can compare the estimated sizes of the two inputs and swap them if needed: `Join(condition, left, right)` becomes `Join(condition, right, left)` when the right is estimated to be smaller. This is a physical-plan rewrite (it runs after the join-order search has chosen which tables to join, but before the executor receives the plan).

6.5 Application Order

Apply the rules in this order:

1. Constant folding (resolves trivial predicates and possibly prunes branches).
2. Predicate pushdown (moves filters as low as possible).
3. Projection pushdown (limits column propagation).
4. Repeat 1–3 until no change (typically 1 or 2 iterations are enough).
5. Run the join-order search (Section 9), which produces a new plan tree.
6. Apply join input swap on the result based on cost estimates.

7 The Cost Model

The cost of a plan is computed bottom-up, with each operator contributing some cost based on the size of its input(s). The convention this project uses is dimensionless: 1 unit per row read or written or processed. Specifically:

7.1 Cardinality Estimation

Each operator emits some number of rows. The estimator computes this number based on the input cardinalities and per-column statistics:

- `Scan(t)`: emits `t.row_count` rows.
- `Filter(pred, child)`: emits `child.cardinality * selectivity(pred)` rows. Selectivity formulas for individual predicates:
  - `col = literal`: $\frac{1}{\texttt{distinct_count(col)}}$. (If the literal is in the min–max range; if outside, selectivity is 0.)

- `col != literal`: $1 - \frac{1}{\texttt{distinct_count(col)}}$.
- `col < literal` or similar range: estimate as the fraction of the min–max range that satisfies the predicate. For integer columns, this is $(\texttt{literal} - \texttt{min})/(\texttt{max} - \texttt{min})$, clamped to $[0, 1]$.
- Multiple conjuncts (`AND`): assume independence and multiply the selectivities. This is the standard assumption and is often wrong for correlated columns, which is why real optimizers also have multi-column statistics. For this project, the simple independence assumption is acceptable.

- `Join(condition, left, right)`: for an equijoin on `left.col1 = right.col2`, the standard estimate is $\frac{\texttt{left.cardinality} \cdot \texttt{right.cardinality}}{\max(\texttt{distinct_count(left.col1)}, \texttt{distinct_count(right.col2)})}$. This is the standard System R formula. For a cross product (no condition), the cardinality is the product. The output column statistics are inherited from the inputs (use the maximum of the two distinct counts when both sides have a column with the same name).
- `Project, Limit, GroupBy`: simple to estimate; defer to the obvious formulas.

7.2 Cost Function

For each operator, the cost is its input cardinalities plus its output cardinality, with a small multiplier based on operator type:

- `Scan(t)`: `t.row_count` (the cost of reading every row).
- `Filter(pred, child)`: `child.cost + child.cardinality` (read every input row, decide).
- `Join(cond, left, right)` as a hash join: `left.cost + right.cost + 2 * left.cardinality + right.cardinality + output.cardinality`. The build-side input is read twice (once to read it, once to hash it); the probe-side is read once; the output is materialized.
- `CrossProduct`: `left.cost + right.cost + left.cardinality * right.cardinality`. This is intentionally enormous to bias the optimizer away from cross products.
- `Project, Limit, GroupBy`: `child.cost + child.cardinality`.

The exact constants do not matter much; what matters is that the cost grows correctly with cardinality so that bad plans (huge intermediate results) are correctly identified as expensive.

7.3 Validating Estimates

Your design document must include an "estimate accuracy" table for the benchmark queries. For each query, run the chosen plan, instrument the executor to record the actual cardinality of each intermediate result, and compare to the estimated cardinality. Report the ratio. A ratio between 0.5 and 2 is good; between 0.1 and 10 is acceptable; outside that range, your estimator is broken and you should investigate before submitting.

8 Join-Order Search

This is the centerpiece of the project. Given a set of n base tables (after pushdown has filtered them) and a set of join conditions between them, find the cheapest left-deep binary join tree that joins all n.

8.1 Why Left-Deep

A binary join tree is *left-deep* if every join's right input is a base table (a single `Scan` or `Filter(Scan)`, never another `Join`). This restricts the search space dramatically: with n tables, there are $n!$ left-deep trees, instead of $\frac{(2(n-1))!}{(n-1)!}$ for general binary trees (which is 30 for $n = 4$ rather than 120). Real optimizers since System R use left-deep trees as the default because they pipeline well and the search is tractable.

For the base project, left-deep only. Bushy join trees (where the right side can also be a join) are a bonus.

8.2 Selinger DP

The classic dynamic programming algorithm, due to Selinger et al. (System R, 1979). The state is a subset of the base tables; the DP value is the cheapest left-deep plan that joins exactly that subset.

1. Represent each subset of the n base tables as a bitmap of n bits. With $n \leq 4$ (or up to 8 in the bonus), an `int` or `long` suffices.
2. For every singleton subset $\{t\}$, the best plan is `Scan(t)` (or `Filter(p, Scan(t))` if a predicate has

been pushed down). Record its cost and cardinality.

3. For every subset S of size ≥ 2, in increasing order of size, iterate over every way to split S into a non-empty left part L and a singleton right part $\{t\}$ with $t \in S$. For each split:
   1. Look up the best plan for L (already computed).
   2. The right side is the base table t with any pushed-down filters.
   3. If there is no join condition between L and t (no predicate connecting any table in L to t), this split is a cross product; skip it unless this is the only option.
   4. Otherwise, compute the cost of `HashJoin(condition, plan(L), Scan(t))`.

   The best plan for S is whichever split has the smallest cost.
4. The answer is the best plan for the full subset $\{t_1, \ldots, t_n\}$.

The total work is $O(n \cdot 2^n)$ to fill the table and $O(n)$ per cell to enumerate splits, so $O(n^2 \cdot 2^n)$. For $n = 4$, this is 64 operations; for $n = 8$, 16,384. The algorithm scales well up to about 12 tables, beyond which memory becomes the limit. This project's queries are limited to $n \leq 4$, so the search is essentially instantaneous.

8.3 Connecting Back to the Plan Tree

After the DP completes, you have the cost of the cheapest plan but not the plan tree itself. Either store the chosen split alongside each subset's cost (so you can reconstruct the tree by walking back), or store the actual best plan tree directly (more memory, simpler code). For four tables, both are fine.

9 The Executor

Same as the simpler version of this project: a materialized model in which each operator is a function that takes input rows and returns output rows, with no state held between calls. The six operators are `Scan, Filter, Project, CrossProduct, HashJoin, Limit`, plus `GroupBy` (which is new for this project).

`GroupBy(col, agg, child)` reads every row from the child, builds a hash map from the value of `col` to a running aggregate state (sum + count for AVG; running min for MIN; etc.), and after the input is exhausted, emits one row per group with the final aggregate value.

The executor is short — about 200 lines of code for all seven operators combined — and is the same code you would write for the simpler version of this project. The reason is that the project is about the optimizer; making the executor sophisticated would not change which plans the optimizer chooses.

10 Phase 1 — Parser, Catalog, and Executor

The goal of Phase 1 is an end-to-end query pipeline with no optimizer at all: queries parse into the naive plan from Section 4, execute correctly, and produce results. The catalog and statistics are loaded but not yet used.

Build:

- The hand-written SQL parser for the grammar in Section 5.
- The logical-plan tree representation.
- The catalog with table schemas and base statistics; load by scanning each CSV once at startup. Cache to `catalog.json`.
- The materialized executor with all seven operators.
- A naive plan builder that turns the parsed query into the unoptimized plan tree (FROM order, all predicates filtered on top of cross products).
- The `qopt` shell with `LOAD`, `QUERY`, `EXPLAIN`, and `\stats` commands.

At the end of Phase 1, every query in the benchmark suite should run and produce the right answer. Two-table queries should be reasonably fast (a few seconds at worst); three- and four-table queries on the benchmark dataset will be very slow because of the cross-product blowup. This is expected and is what Phase 2 will fix.

11 Phase 2 — The Rule Rewriter and the Cost Model

The goal of Phase 2 is to add the four rewrite rules and the cost model. Join-order search comes in Phase 3.

Build:

- The four rules from Section 7: predicate pushdown, projection pushdown, constant folding, join input swap. Each is a recursive function that walks the plan tree and returns a transformed tree.
- The fixed-point loop that applies them in order until none changes the plan.
- The cost model from Section 8: cardinality estimates for each operator, the cost function, and a top-level `cost(plan)` function.
- Update `EXPLAIN` to print the chosen plan, its estimated cardinality at each operator, and the total estimated cost.

At the end of Phase 2, two-table queries with selective filters should be dramatically faster than in Phase 1 because predicate pushdown filters the base tables before the join. Three- and four-table queries are also faster, but join-order is still the FROM order; Phase 3 will fix that.

11.1 Validation

For each two-table query in the benchmark, verify by hand that the chosen plan applies pushdown correctly. Instrument the executor to log the actual cardinality at each operator. Compare to the estimated cardinality. The ratio should be within 2× for selective predicates; if it is wildly off (10× or more), there is a bug in the cost model.

12 Phase 3 — Join-Order Search and Benchmark

The goal of Phase 3 is the Selinger DP and the benchmark.

Build:

- The bitmap subset enumeration: iterate subsets in size order.
- The DP table: subset → (cost, cardinality, best plan).
- The split enumeration for each subset: try every singleton on the right.
- The connectedness check: skip splits that would produce a cross product.
- Plan reconstruction from the DP table.
- Integration: after rule rewriting, extract the set of base tables and join conditions from the plan, run the DP, and replace the plan with the result.
- The benchmark suite (see below).

12.1 The Benchmark Dataset

A four-table schema (a small TPC-H-like sales scenario):

- `customers` (10,000 rows): id, name, country (24 distinct), age.
- `orders` (500,000 rows): id, customer_id, total, year (5 distinct), status.
- `line_items` (2,000,000 rows): order_id, product_id, qty, price.
- `products` (50,000 rows): id, name, category (87 distinct), supplier_id.

You generate this data deterministically from a fixed seed using a small CSV generator program. The exact distributions are up to you, but document them.

12.2 The Benchmark Queries

Run these queries against four optimizer configurations: (1) no optimization, (2) rules only, (3) DP only (no rules), (4) full optimizer (rules then DP). For each query and configuration, report the estimated cost, the actual runtime, and the actual result cardinality. The required queries:

1. **Q1 (two-table, selective):** `SELECT * FROM customers, orders WHERE customers.id = orders.customer_ AND customers.country = 'PK'`. Expected: full optimizer beats no-optimizer by at least 100×; rules alone get most of the win; DP alone helps less because there are only two tables.
2. **Q2 (three-table, selective):** the example from Section 2. Expected: full optimizer wins by at least 100×; both rules and DP contribute.

3. **Q3 (four-table, selective):** `SELECT customers.name, products.name FROM customers, orders, line_items, products WHERE customers.id = orders.customer_id AND orders.id = line_items.order_ AND line_items.product_id = products.id AND customers.country = 'PK' AND products.category = 'Electronics'`. Expected: full optimizer wins enormously; suboptimal join orders cause intermediate explosions.
4. **Q4 (aggregation):** `SELECT customers.country, SUM(orders.total) FROM customers, orders WHERE customers.id = orders.customer_id AND orders.year = 2024 GROUP BY customers.country`. Expected: rules win (predicate pushdown on `year`).
5. **Q5 (adversarial):** a query specifically constructed to trip up a naive optimizer. Multiple selective filters but on the join's larger side, so that filtering early makes the difference. The full optimizer should still win.

The benchmark output goes in the design document as a 5×4 table of speedups, plus a discussion of which component (rules vs. DP) contributes how much per query.

12.3 Estimate Accuracy Table

For Q3 specifically (the four-table query), report the estimated and actual cardinality at every intermediate result in the chosen plan. This is the table real database engineers use to debug bad cost models, and you will use it to identify any large estimation errors in your model.

13 Deliverables and Submission

13.1 What to Submit

A single zip archive containing:

- Complete source tree of the optimizer (`qopt`), the CSV data generator, and the benchmark driver.
- A `Makefile` (or `CMakeLists.txt`) that builds everything with one command on a recent Ubuntu LTS.
- A `README.md` with build instructions, supported queries, and limitations.
- A `design.pdf` of at most 13 pages describing: the six-component architecture; the parser and grammar; the catalog and statistics; each of the four rewrite rules with a worked example; the cardinality estimation formulas including the System R join formula; the cost function; the Selinger DP algorithm and how splits are enumerated; the executor's seven operators; and a detailed discussion of the benchmark results including the estimate-accuracy table for Q3.
- A `benchmark/` directory with the data generator, the benchmark driver, and the output of your final run.
- A `tests/` directory with unit tests for: the parser, each of the four rewrite rules in isolation, the cardinality estimator (verify formulas against hand-computed examples), the Selinger DP (verify on a 3-table example with known optimal plan), and end-to-end query correctness.

13.2 Archive Naming

`Group[Number]_Project02_Optimizer.zip`, for example `Group17_Project02_Optimizer.zip`.

13.3 Demonstration

Each group will present a 25-minute live demonstration. First, a walkthrough of the optimizer pipeline using the design document, with particular attention to the Selinger DP. Second, a live execution of the benchmark, with the instructor free to type new queries on the spot and inspect the chosen plans via `EXPLAIN`. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component, with particular focus on the cost model formulas (especially the System R join cardinality formula) and on why predicate pushdown is correct (why pushing a filter through a join does not change the result). All group members must be present.

14 Bonus Opportunities

Each independent. Up to 20 percentage points total.

14.1 Bushy Join Trees (up to 5%)

The base project uses left-deep trees only. Extend the DP to consider bushy trees: for each subset S, instead of splitting into $L \cup \{t\}$, enumerate all splits $L \cup R$ where both L and R are non-empty subsets of

S. The search space and runtime grow significantly, but bushy trees can be much better than left-deep when intermediate results are unbalanced. Demonstrate at least one query where bushy beats left-deep by a measurable margin.

14.2 Histograms for Range Selectivity (up to 10%)

Replace the simple min–max range estimate for range predicates with equi-depth histograms. Each numeric column gets a histogram of, say, 32 buckets, where each bucket holds approximately the same number of rows. Range selectivity is then computed by summing the buckets that fall within the range. This dramatically improves accuracy on skewed distributions. The benchmark must include a query with a skewed distribution where histograms produce a noticeably better plan than the base estimator.

14.3 Sort-Merge Join as an Alternative (up to 5%)

The base project uses hash join exclusively. Add sort-merge join as a second physical join algorithm. The optimizer chooses between hash and sort-merge based on cost (sort-merge is typically better when both inputs are already sorted on the join key, which can happen if a previous operator produced sorted output). Discuss the cost model for sort-merge in the design document.

15 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing SQL parser, query optimizer, or database engine as a component of your implementation. Includes SQLite, DuckDB, libpg_query, ANTLR-generated parsers for SQL, and bindings.
- Use of any existing parser-generator (yacc, bison, ANTLR). The parser must be hand-written recursive descent.
- Use of any existing binary serialization library for the catalog or any other state.
- Hard-coding the join order based on table size or schema knowledge instead of running the DP. The optimizer must produce its plan via the cost model and search.
- Producing wrong query results. The chaos test will run every benchmark query and verify the result against an independent reference (the unoptimized baseline). Plans that produce wrong rows fail the project.
- Use of Python at any layer, including the data generator and the benchmark driver. Tests and scripts must be in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits.

16 Required Reading

1. Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., and Price, T. G. "Access Path Selection in a Relational Database Management System." *ACM SIGMOD*, 1979. The paper that introduced cost-based join ordering. Sections 3 and 4 describe exactly the algorithm you will implement. **Required reading before Phase 3**; the manual you are reading now is a summary, but the paper is the original.
2. Graefe, G. "Query Evaluation Techniques for Large Databases." *ACM Computing Surveys*, 1993. A comprehensive survey of query execution and optimization. Sections on join algorithms (4) and on optimizer architecture (10) are the relevant ones. Reference material; skim it during Phases 1 and 2.
3. Ramakrishnan, R. and Gehrke, J. *Database Management Systems*, 3rd edition, Chapter 12 ("Overview of Query Evaluation") and Chapter 14 ("A Typical Relational Query Optimizer"). The textbook treatment of everything in this manual. Read 12.1 and 14.1 before Phase 1; 14.2–14.4 before Phase 3.
4. Leis, V. et al. "How Good Are Query Optimizers, Really?" *Proceedings of the VLDB Endowment*, 2015. A modern empirical study of how cardinality estimation errors propagate through plans. Optional but illuminating; explains why your cost model's accuracy matters so much.
5. Pavlo, A. *CMU 15-445 Database Systems*. Lectures 12–14 cover query execution; lectures 14–16 cover optimization. Freely available on the CMU course website.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM